Carbon Emission Calculator

Packaging & Transport Emissions Analysis Tool

Attribute	Value
Built with	Python, Streamlit, SQLite, Pandas, NumPy
AI / ML	None - formula-based engine (FEFCO industry standards)
Database	SQLite - local, zero setup, auto-created on first run
Deployment	.bat file or Streamlit server
Standards	FEFCO packaging standards, published CO2 emission factors

1. What This Project Is

Atlas Copco needed a practical way to quantify the carbon emissions generated by their product packaging and logistics operations. The question they were trying to answer was straightforward: given a product of a certain size, what materials should we use to package it, and how do our transport choices affect total CO2 output?

The calculator gives operations staff a way to enter product dimensions, choose packaging materials, select a transport mode, and immediately see the estimated CO2 emissions. Both a design path (calculated from geometry) and a physical path (from known weights) are supported.

The tool was built to replace manual Excel work with something that persists records, is usable by non-technical staff, and produces auditable, timestamped results.

Scope of Work

The project involved three distinct areas of responsibility:

1. Translating the Excel-based FEFCO formula sheet into an accurate, validated Python calculation engine
2. Building end-to-end database integration to capture inputs, persist records, and allow retrieval and filtering
3. Making the app genuinely usable without any technical setup - no terminal, no IDE, no configuration, via a bat file.

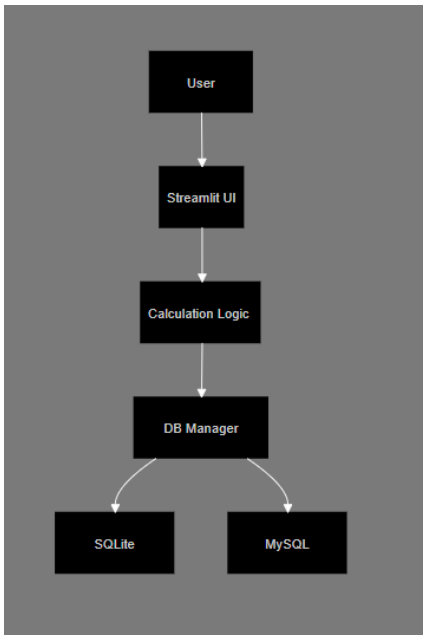

Figure 1. System Architecture

2. How the Calculations Work

The engine has two parallel paths. The design path takes product dimensions and derives packaging weights from geometry. The physical path takes weights entered directly by the user. Both paths produce CO2 values. The app shows which configuration is more sustainable.

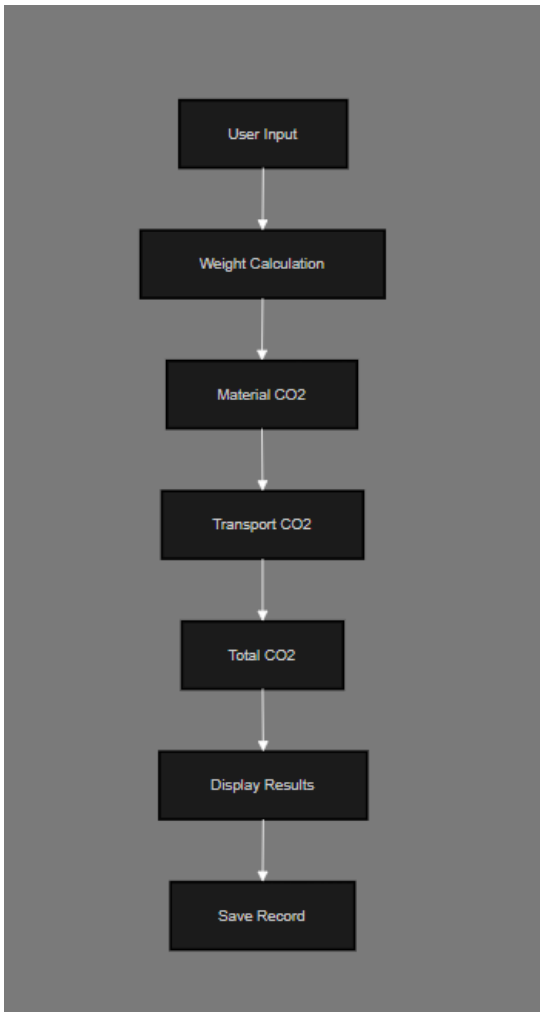

Figure 2. Application Workflow

2.1 The 40mm Clearance Buffer

Every calculation begins by adding 40mm to each product dimension (length, width, height). This clearance accounts for the space needed to fit the product inside the box, include cushioning material, and allow flaps to close properly. It is a standard assumption in packaging engineering.

A product of 600 x 400 x 300mm results in a packaging box of 640 x 440 x 340mm. All subsequent calculations use these larger dimensions.

2.2 Corrugated Box: Area and Weight

The surface area of the corrugated box is calculated from the buffered dimensions using the standard rectangular surface area formula. The result is then adjusted by a ply factor, because corrugated board is not a flat sheet. It is a layered structure: fluted paper between liners. More plies means more material and a higher effective area.

Ply	Adjustment Applied	Reason
3-ply	Base area + 170% of base	Single wall, one flute layer
5-ply	Base area + 110% of base	Double wall, two flute layers
7-ply	Base area + 105% of base	Triple wall, three flute layers

Weight is derived from the adjusted area with a further ply-specific multiplier (3-ply reduces by 25%, 5-ply increases by 5%, 7-ply increases by 45%). These factors are sourced from FEFCO, the European Federation of Corrugated Board Manufacturers.

2.3 Wooden Box: Hollow Volume Method

A wooden box is a hollow shell, not a solid block. The calculation subtracts the interior void from the total outer volume, using the wall thickness entered by the user. The net volume in cubic metres is multiplied by 600 kg/m3, the standard density of structural timber (pine and spruce), to get the weight.

2.4 Pallet: Three-Component Structure

A standard pallet is modelled as three separate components: a deck board on top, nine runner blocks underneath (a 3x3 grid, consistent with EUR/EPAL pallet specification), and three planks along the base. The deck and planks scale with the product dimensions. The runners are fixed at 125 x 110 x 90mm. Total weight uses a density of 500 kg/m3, reflecting typical pallet-grade softwood.

2.5 Material CO2 Emissions

Each material type has a published emission factor representing the CO2 released to manufacture one kilogram of that material. Emissions are calculated as weight multiplied by the factor.

Material	Emission Factor
Corrugated board	0.491 kg CO2 per kg
Solid wood	0.31 kg CO2 per kg
Plywood	0.68 kg CO2 per kg

2.6 Transport CO2 Emissions

Transport emissions are calculated as total weight in tonnes multiplied by distance in kilometres, multiplied by the emission factor for the selected mode. The difference between modes is substantial.

Transport Mode	Emission Factor	Relative to Sea
Road	0.062 kg CO2 / tonne-km	~6x
Rail	0.022 kg CO2 / tonne-km	~2x
Sea freight	0.016 kg CO2 / tonne-km	baseline
Air freight	0.610 kg CO2 / tonne-km	~38x

Air freight is dramatically more carbon-intensive because aircraft burn far more fuel per unit of cargo weight, and the warming effect of emissions at altitude is amplified. The calculator makes this difference explicit and quantifiable for decision-makers.

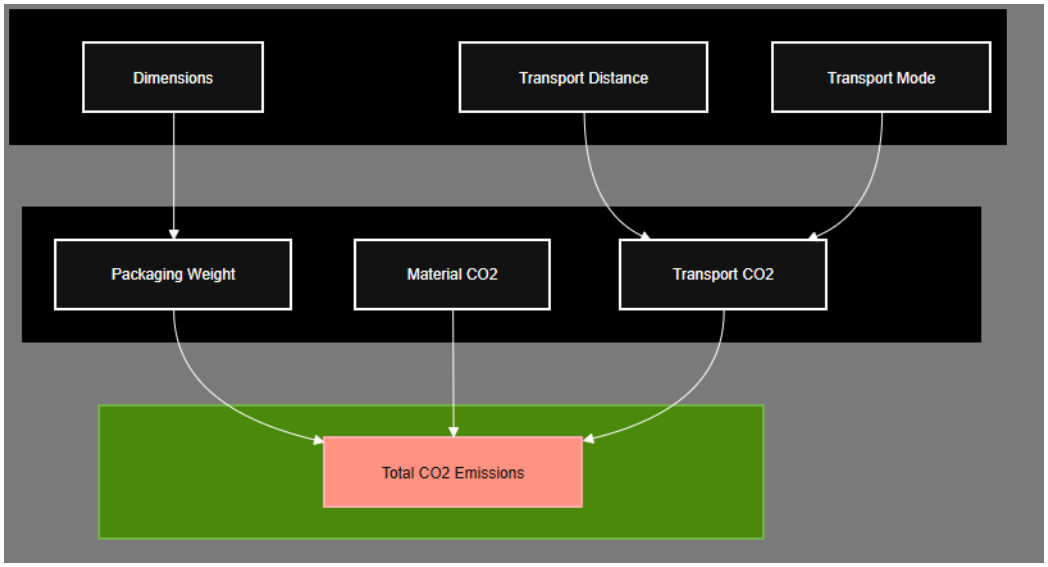

Figure 3. Carbon Calculation Engine

3. Database Design

Why SQLite

SQLite was chosen deliberately. This is a single-user, local desktop tool. There are no concurrent users, no network required, and no server to maintain. SQLite is built into Python, creates itself automatically on first run, and stores everything in a single portable file. For this scope, it is the appropriate choice.

If the tool were scaled to serve multiple Atlas Copco plants simultaneously, a server-based database like PostgreSQL or MySQL would be the right move. The code already includes a MySQL configuration path for exactly this scenario.

Schema

Records are stored in a single table. Input fields and computed results are serialised as a JSON object and stored in a text column. This means the schema can evolve without requiring database migrations as the tool is extended.

Column	Type	Contents
id	Integer (auto)	Primary key
input_json	Text	All user-entered fields as a JSON object
output_json	Text	All calculated results as a JSON object
description	Text	Optional note entered by the user
timestamp	Text	Date and time from the OS at the moment of calculation

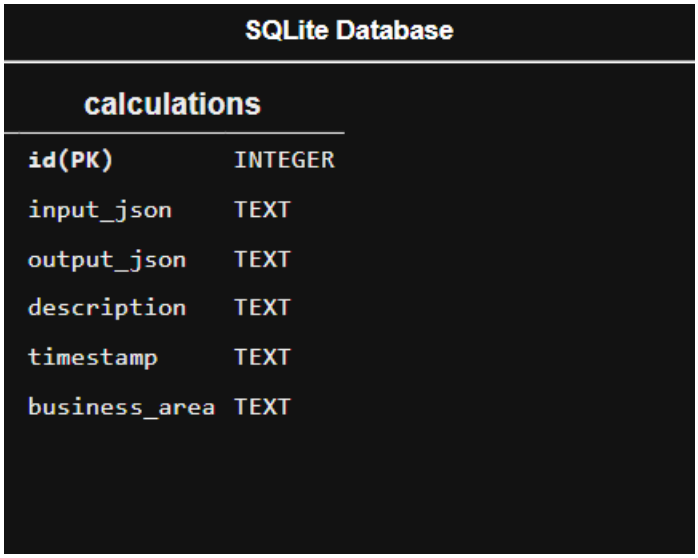

Figure 4. Database Schema

Filtering Logic

Because input fields are stored inside a JSON column rather than as native SQL columns, standard SQL WHERE clauses cannot filter on them directly. The current approach retrieves all records and filters in Python. This is clean to develop and perfectly adequate for the current data volume.

The filter options displayed to the user are derived from the actual data rather than hardcoded. If the first record was entered in April 2026, only April 2026 appears as an option. This also makes the filter panel act as a natural indicator of when the tool started being used.

4. Architecture and Request Flow

There is no AI model or external API involved in a calculation. All calculations are performed locally using deterministic engineering formulas and published emission factors.

Step	What Happens
1. Input	User fills the Streamlit form: product name, dimensions, material type, transport mode, distance, and an optional note.
2. Dimension adjustment	40mm is added to each dimension to produce the actual packaging box dimensions.
3. Weight calculation	Box volume is computed. Material density (from FEFCO data by ply count) gives the packaging weight in kg, converted to tonnes.
4. Emission calculation	CO2 (kg) = weight in tonnes x distance in km x transport emission factor.
5. Persist	All inputs and results are serialised as JSON and inserted into SQLite.
6. Display	Streamlit renders the result card showing CO2 breakdown and any comparison between design and physical paths.
7. Filter and recall	A SELECT query retrieves all records. Python parses the JSON fields, applies the selected filters, and Streamlit renders the filtered table.

Technology Choices

Technology	Role
Streamlit	Frontend UI framework. Renders the form, results, and filter panel. Reruns the entire script on each user interaction.
Python sqlite3	Built-in library. Direct SQL queries for INSERT and SELECT, no ORM layer.
SQLite	File-based relational database. Auto-created. Zero configuration.
Pandas	Used for tabular display of filtered records. Lazily imported for startup performance.
NumPy	Used in weight and volume arithmetic. Lazily imported.
Windows Batch Script	Windows Batch Script

5. Deployment Options

Single Workstation

Single Workstation

The application is launched using a Windows batch (.bat) file. The batch script automatically starts the Streamlit server and opens the application in the user's default browser.

No manual terminal commands are required, making the tool accessible to non-technical users.

Shared Team Tool (MySQL)

For a team where multiple users need to share a central database, the configuration file is updated to point at a MySQL server. The app creates the database and table automatically on first connection. It does not need to run any SQL scripts manually. The same .exe is then rebuilt and distributed to all users.

Validation

The formula engine was validated against known Excel reference outputs before release. The test inputs below should produce the values listed exactly.

Output	Expected Value	Source
Corrugated box area	2.724960 m2	Excel M16
Corrugated box weight	2.861208 kg	Excel N16
Wooden box volume	0.012416 m3	Excel M19
Wooden box weight	7.449600 kg	Excel N19
Pallet volume	0.023651 m3	Excel M22
Pallet weight	11.825550 kg	Excel N22
Transport CO2 (Air, 1000km)	15.058922 kg CO2	Backup!H32

Test inputs: L=600mm, W=400mm, H=300mm, 5-ply, wall thickness 20mm, solid wood, plywood pallet, air transport, 1000km.

6. What This Project Does Not Include

Some technologies that might be expected are deliberately absent. This section explains why.

Technology	Why Absent?
AI or LLMs	The calculation is deterministic. No natural language, no pattern recognition, no ambiguity. An LLM would add complexity with no functional benefit.
Embeddings or vector database	All data is structured and numerical. Embeddings are for semantic search over unstructured text, which is not applicable here.
Authentication	Out of scope for this version of the software
st.cache_data	Identified as a future improvement. Without it, the database is re-queried on every Streamlit interaction even when the data has not changed.
Cloud storage like S3	Due to requirements of storing data only on secure machines/servers that are owned by the company, it was decided to use local storage mechanisms to store this data.

7. Key Technical Learnings

• Translating engineering formulas from Excel into Python
• Building a maintainable calculation engine
• Working with SQLite.
• Designing software for non-technical business users
• Evaluating trade-offs between schema flexibility and query performance

8. Known Improvements for Future Versions

1. Move filtering logic into SQL using json_extract() to avoid loading all records into Python memory on each request.
2. Add st.cache_data to the database fetch function to eliminate redundant queries on UI interactions.
3. PDF or Excel export for operations teams that need printable reports.